

SPRAWOZDANIE

Zajęcia: Matematyka Konkretna

Prowadzący: prof. dr hab. inż. Vasyl Martsenyuk

Laboratorium Nr 1 Data 04.12.2025 Temat: "Algorytm LSTM dla tekstu" Wariant 10	Jakub Janik Informatyka II stopień, stacjonarne, 2 semestr, gr. CB
--	---

1. Cel ćwiczenia:

Celem ćwiczenia było zaimplementowanie i wytrenowanie rekurencyjnej sieci neuronowej typu LSTM (Long Short-Term Memory) za pomocą metody BPTT (Backpropagation Through Time). Trening miał na celu nauczenie modelu przewidywania następnego znaku w sekwencji na podstawie dostarczonego tekstu.

2. Przebieg ćwiczenia:

Wprowadzone dane:

Parametr	Wartość	Opis
Rozmiar warstwy ukrytej	50	Pojemność pamięci i złożoność modelu
Liczba kroków czasowych	50	Długość sekwencji używanej do propagacji wstecznej
Dane wejściowe	Example.txt	Plik z danymi w tym przypadku wpisane są na sztywno
Współczynnik uczenia	Le^{-1}	Szybkość aktualizacji wag
Metoda optymalizacji	AdaGrad	Używana do adaptacyjnej aktualizacji wag

Konfiguracja Hyperparametrów Wariantu 10:

- Wartości stałych H_size (rozmiar warstwy ukrytej) oraz T_steps (długość sekwencji BPTT) zostały zdefiniowane zgodnie z Wariantem 10

Użycie Danych Zastępczych:

- W związku z błędem, mechanizm wczytywania danych z pliku został zastąpiony przez bezpośrednie wklejenie treści do zmiennej data. Działanie to pozwoliło na kontynuację treningu przy zachowaniu statystyki znaków podobnej do oczekiwanej dla plików źródłowych

Poprawa Kompatybilności Bibliotek:

- W celu zwiększenia stabilności kodu i poprawy czytelności konsoli, wszystkie instancje przestarzałej funkcji NumPy np.row_stack zostały zastąpione przez zalecany standard np.vstack, co wyeliminowało błędy typu DeprecationWarning

Kod źródłowy:

```
import numpy as np
import matplotlib.pyplot as plt
from IPython import display
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

try:
    plt.style.use('seaborn-v0_8-white')
except:
    plt.style.use('default')

# Odczytywanie i przetwarzanie danych (Wklejony przykładowy tekst)
data = """
Artificial intelligence (AI) is intelligence—perceiving, synthesizing, and inferring information—demonstrated by machines, as opposed to intelligence displayed by animals and humans. Example of AI are chatbots, autonomous vehicles, and deep neural networks. Deep learning has dominated the field, and this technique has proved highly successful, helping to solve many challenging problems throughout industry and academia. The various sub-fields of AI research are centered around particular goals and the use of particular tools. The traditional goals of AI research include reasoning, knowledge representation, planning, learning, natural language processing, perception, and the ability to move and manipulate objects. General intelligence (the ability to solve an arbitrary problem) is among the field's long-term goals. Computer scientists and philosophers have since suggested that AI may become an existential risk to humanity if its rational capacities are not steered towards beneficial goals.
"""

chars = list(set(data))
data_size, X_size = len(data), len(chars)
char_to_ix = { ch:i for i,ch in enumerate(chars) }
ix_to_char = { i:ch for i,ch in enumerate(chars) }
```

```

# Stałe i hiperparametry (Wariant 10: H_size=50, T_steps=50)
H_size = 50
T_steps = 50

learning_rate = 1e-1
weight_sd = 0.1
z_size = H_size + X_size

# Funkcje pomocnicze
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def dsigmoid(y):
    return y * (1 - y)

def dtanh(x):
    return 1 - x * x

# Inicjalizacja wag
Wf = np.random.randn(H_size, z_size) * weight_sd + 0.5
Wi = np.random.randn(H_size, z_size) * weight_sd + 0.5
Wc = np.random.randn(H_size, z_size) * weight_sd
Wo = np.random.randn(H_size, z_size) * weight_sd + 0.5
Wv = np.random.randn(X_size, H_size) * weight_sd
bf = np.zeros((H_size, 1))
bi = np.zeros((H_size, 1))
bc = np.zeros((H_size, 1))
bo = np.zeros((H_size, 1))
bv = np.zeros((X_size, 1))

parameters = {'Wf': Wf, 'Wi': Wi, 'Wc': Wc, 'Wo': Wo, 'Wv': Wv,
              'bf': bf, 'bi': bi, 'bc': bc, 'bo': bo, 'bv': bv}

smooth_grads = {param: np.zeros_like(parameters[param]) for param in parameters}

# Funkcja Forward Pass
def forward_pass(inputs, targets, h_prev, c_prev):
    xs, hs, cs, zs, os, ys, ps, fs, is_, cs_bar, os_gate = {}, {}, {}, {}, {}, {}, {}, {}, {}, {}, {}
    hs[-1] = np.copy(h_prev)
    cs[-1] = np.copy(c_prev)
    loss = 0

    for t in range(len(inputs)):
        xs[t] = np.zeros((X_size, 1))
        xs[t][inputs[t]] = 1

```

```

zs[t] = np.vstack((hs[t-1], xs[t]))

fs[t] = sigmoid(np.dot(Wf, zs[t]) + bf)
is_[t] = sigmoid(np.dot(Wi, zs[t]) + bi)
cs_bar[t] = np.tanh(np.dot(Wc, zs[t]) + bc)
os_gate[t] = sigmoid(np.dot(Wo, zs[t]) + bo)

cs[t] = fs[t] * cs[-1] + is_[t] * cs_bar[t]
hs[t] = os_gate[t] * np.tanh(cs[t])

os[t] = np.dot(Wv, hs[t]) + bv
ys[t] = np.exp(os[t]) / np.sum(np.exp(os[t]))
loss += -np.log(ys[t][targets[t], 0])

return loss, fs, is_, cs_bar, os_gate, cs, hs, xs, ys

# Funkcja Backward Pass
def backward_pass(targets, fs, is_, cs_bar, os_gate, cs, hs, xs, ys):
    dWf, dWi, dWc, dWo, dWv = np.zeros_like(Wf), np.zeros_like(Wi),
    np.zeros_like(Wc), np.zeros_like(Wo), np.zeros_like(Wv)
    dbf, dbi, dbc, dbo, dbv = np.zeros_like(bf), np.zeros_like(bi),
    np.zeros_like(bc), np.zeros_like(bo), np.zeros_like(bv)

    dh_next = np.zeros((H_size, 1))
    dc_next = np.zeros((H_size, 1))

    grads = {'Wf': dWf, 'Wi': dWi, 'Wc': dWc, 'Wo': dWo, 'Wv': dWv,
             'bf': dbf, 'bi': dbi, 'bc': dbc, 'bo': dbo, 'bv': dbv}

    for t in reversed(range(len(targets))):
        dy = np.copy(ys[t])
        dy[targets[t]] -= 1

        dWv += np.dot(dy, hs[t].T)
        dbv += dy

        dh = np.dot(Wv.T, dy) + dh_next

        dos_gate = dh * np.tanh(cs[t])
        dos_gate = dsigmoid(os_gate[t]) * dos_gate

        dc = dh * os_gate[t] * dtanh(np.tanh(cs[t])) + dc_next

        dfs = dc * cs[t-1]
        dfs = dsigmoid(fs[t]) * dfs

        dis = dc * cs_bar[t]

```

```

        dis = dsigmoid(is_[t]) * dis

        dcs_bar = dc * is_[t]
        dcs_bar = dtanh(cs_bar[t]) * dcs_bar

        zs = np.vstack((hs[t-1], xs[t]))

        dWf += np.dot(dfs, zs.T)
        dbf += dfs
        dWi += np.dot(dis, zs.T)
        dbi += dis
        dWc += np.dot(dcs_bar, zs.T)
        dbc += dcs_bar
        dWo += np.dot(dos_gate, zs.T)
        dbo += dos_gate

        dh_next = np.dot(Wf[:, :H_size].T, dfs) + \
                    np.dot(Wi[:, :H_size].T, dis) + \
                    np.dot(Wc[:, :H_size].T, dcs_bar) + \
                    np.dot(Wo[:, :H_size].T, dos_gate)

        dc_next = dc * fs[t]

    for dparam in grads.values():
        np.clip(dparam, -5, 5, out=dparam)

    return grads, hs[-1], cs[-1]

# Funkcja aktualizacji wag
def update_params(grads, params, smooth_grads):
    for param in grads:
        g = grads[param]
        smooth_grads[param] += g * g
        params[param] -= learning_rate * g / (np.sqrt(smooth_grads[param]) +
1e-8)

# Generowanie tekstu
def sample(h_prev, c_prev, seed_ix, n):
    x = np.zeros((X_size, 1))
    x[seed_ix] = 1
    ixes = []

    h = h_prev
    c = c_prev

    for t in range(n):
        z = np.vstack((h, x))

```

```

        f = sigmoid(np.dot(Wf, z) + bf)
        i = sigmoid(np.dot(Wi, z) + bi)
        c_bar = np.tanh(np.dot(Wc, z) + bc)
        o = sigmoid(np.dot(Wo, z) + bo)

        c = f * c + i * c_bar
        h = o * np.tanh(c)

        out = np.dot(Wv, h) + bv
        p = np.exp(out) / np.sum(np.exp(out))

        ix = np.random.choice(range(X_size), p=p.ravel())

        x = np.zeros((X_size, 1))
        x[ix] = 1
        ixes.append(ix)

    return ixes, h, c

# Główna pętla treningowa
def training_loop():
    global Wf, Wi, Wc, Wo, Wv, bf, bi, bc, bo, bv

    h_prev = np.zeros((H_size, 1))
    c_prev = np.zeros((H_size, 1))

    n, p = 0, 0
    moving_avg_loss = -np.log(1.0/X_size) * T_steps

    while n <= 10000:
        if p + T_steps + 1 >= data_size or n == 0:
            h_prev = np.zeros((H_size, 1))
            c_prev = np.zeros((H_size, 1))
            p = 0

        inputs = [char_to_ix[ch] for ch in data[p:p + T_steps]]
        targets = [char_to_ix[ch] for ch in data[p + 1:p + T_steps + 1]]

        loss, fs, is_, cs_bar, os_gate, cs, hs, xs, ys = forward_pass(inputs,
targets, h_prev, c_prev)

        moving_avg_loss = moving_avg_loss * 0.999 + loss * 0.001

        grads, h_prev, c_prev = backward_pass(targets, fs, is_, cs_bar,
os_gate, cs, hs, xs, ys)

        update_params(grads, parameters, smooth_grads)

```

```

    if n % 500 == 0:
        seed_ix = inputs[0] if inputs else char_to_ix[chars[0]]
        sample_ix, _, _ = sample(h_prev, c_prev, seed_ix, 200)
        txt = ''.join(ix_to_char[ix] for ix in sample_ix)

        print(f'Iteracja: {n}, Strata: {moving_avg_loss:8.4f}')
        print('--- GENEROWANY TEKST ---')
        print(txt)
        print('-----')

    p += T_steps
    n += 1

    print(f'Trening zakończony po {n-1} iteracjach. Ostateczna strata: {moving_avg_loss:8.4f}')

# Uruchomienie pętli treningowej (GŁÓWNY KROK)
print("Rozpoczynanie treningu dla Wariantu 10 (H_size=50, T_steps=50, dane: Wklejony tekst)...")
training_loop()

```

Wyniki:

Rozpoczynanie treningu dla Wariantu 10 (H_size=50, T_steps=50, dane: Wklejony tekst)...

Iteracja: 0, Strata: 186.8835

--- GENEROWANY TEKST ---

CltEmwCoGv cp(-a lc,xI.lri(fgzIniIk

Ir i(rhg-D(,jDrzvtA-IrvDnstnEccvzulll,idi.(nmirior tysAgiCIv cI-
du())EGklGnTbLin,yo,rhAtr)Ihn)Irnlkfk(cnacxmnlcetzo)))gi sri e-i)ecGgxcj.iET-
ttl Eysipclt-Ar(iwigihck

Iteracja: 500, Strata: 173.3460

--- GENEROWANY TEKST ---

m'ttImilii gtliiiifticttibgtfgitsnnregtgtunsgelIett crnet(itt ian iincty
ntttcgcilcm ilttyleAr nu uttiytktctgiatAtigtgt inc i ecao aIu ursctiieitci-
ledh tiietthgiItgrscstt lasgxs.aicnt-icto)sc-moitrtfa

Iteracja: 1000, Strata: 162.6485

--- GENEROWANY TEKST ---

mdrrdgi e fhi uttaon flrredhylheemberngn sfe.phas hespmlsc locsnecnof d me
atolit lpbabvilulydahn dssle ame b nbar l elben nhaie-
neeul,arrantsovspcnegictddynoisdsIgs ayaebaaliargbldndeuloosenhnnnn

Iteracja: 1500, Strata: 155.7347

--- GENEROWANY TEKST ---

snIlEIietr.-,griIitEh-triGnsitp.TtEE.tt-.Giuit..r lnq.i.tleGr rrrhirt
.rltiItCmirtrttsrThuiECih.

tqEE,.qinrgIrliEIhrr.rristrrreriuiEElitrft,i.cti'tiE.. .tEr
tn.T.,Tehm.rphtrtvEaviTt.Gn.r.niT,h Ta's'G

Iteracja: 2000, Strata: 151.1680

--- GENEROWANY TEKST ---

Inc aseeesaeeee eee-m.rr.hT ee
nreeeio,keseuett .teGriKh.rraieeine teei)ia,.u,,ut e.T.Tafv-etdnsTtad set v
rgtsi f rprtc eaelp rrheoA tAar taua rarlreeano snnroe peo uhreeeee-
etoeeeekeeeeeee,e rete

Iteracja: 2500, Strata: 147.7158

--- GENEROWANY TEKST ---

ned ,ren lnosnt inr r ttitomhnnysaoe i alrnsiltvts lAl iitntztgi(ieet ose-
ti,d uarpmmt g Afc r f lg lamvte et ml p eu r ueitettee s eiaai et tl
teea epiytgtgrlt nanlvl npnnlrtrlrpevlprtt

Iteracja: 3000, Strata: 145.3468

--- GENEROWANY TEKST ---

oesn mi no sig lntorsne lncsit ats l aaiositetlsnat ieaa tneosdvact
nbl(atrae nleci ymsvra eli m noa yeaspl c aavl ilsy estm ate tsurrranl ie
tlg(itlnodovamhltm eetpe aeaaoi nnrAd na -tntonoc

Iteracja: 3500, Strata: 143.9175

--- GENEROWANY TEKST ---

nryieltsytori ijfpyaastbtr gei, t -st tt,tgaeru ab ta etjat ktrteaaiuares-
sebollni aebbti(misehhlh(u.hht.ihe gif(lauge-hhehr riAt bhr
htuiiAhhrhhhrboenn retra m .emesemmeoeueho attee ot teeeme oit

Iteracja: 4000, Strata: 142.7248

--- GENEROWANY TEKST ---

eas iAoieypeae m ,e, eaoa n annn u)sieiIluppho ity-
hjhihhhhhhhhhhhhhhhhhhhh.oqihhhheahrrhhhhhmaai aoeraeqaerrherotqaitqaasAat-
aAaAttaAranketaetGtAeA pp eetetye eonnnnnnnneaneoeebd uta ulaan nnn ,t

Iteracja: 4500, Strata: 141.6329

--- GENEROWANY TEKST ---

cTsnbhtcm (telta att atooa-eyp)atlaat
alo)GaphsaasxxxntlvaxagrgTTajTggTTigTATjaxianTriaTiAaal alaaaaAipaajyap ylna-
rAaeipyAr nrtenrnnfaenapniaaannai nrarnnnnnraa rn n nac rrrrr accancppn-
ra,rnrassnnr a

Iteracja: 5000, Strata: 140.7427

--- GENEROWANY TEKST ---

ingenArhaler(tyeg,ioete a, eeb, obaasgaaenvsfta, drslmrrmtlnlnlnlIhlnnla
rcthaanlbnhhahxrrmrxxnclynTTglggpgTsgrttTthmau rasaramtaaauiameDapanaAAegaeia-
ATar aaa.aaaimpaxpAAaa aanuxuientnatnnetnantnnnnn

Iteracja: 5500, Strata: 140.2136

--- GENEROWANY TEKST ---

cigpczm leslaaintapno. te,rttonaa abauahedtmtnnannnlrrtxtAnnAvAAAvflnnlllxxal-
navnnDlGnnanangcx lnlllgytxgilTDT(vtecTgccxGg(AeiaTlebn. aaaxnt.snaanansnn
na a nraaaonaenan tcnlealeaeaana neeanaieseaea

Iteracja: 6000, Strata: 139.6750

--- GENEROWANY TEKST ---

Gerrg catey mcaltntaatg te,e,wyt ,,t,eaiaah aeltaalnacyaaacgihtha-
euhvjihmsshoharelhohehnha helhhherhrnhhhhehchirhrmhihhhlhnneuhehrrrelrrhy-
hheherkaylhheuhel h Gusetmhasroenhlraehusectnhhlvhhrhhoheh

Iteracja: 6500, Strata: 140.1243

--- GENEROWANY TEKST ---

dn l l urnloolosonaa T feageigaosdTgallva lags,rr-aggbc geA gtne ttoirttmee-
nantpen na, nataagro ,a,calettaanyanaranaaaa cag)g eggi aggh honaaamla
ataoeaagirsoaegeop) eean oetaugnaae)c aonoucae gaataa

Iteracja: 7000, Strata: 140.1505

--- GENEROWANY TEKST ---

dtsnen cnytlczoaiiplaeaidtotdcctlgcctci teoarna gn,ate ,aotadtea,aba Aaea
tazaaunaa e aaamapaa utla oduoehaeap phe eomiaaauurvaIeeeietaua nateaoga-
afduaeeee ghiaa aeahuaeeh pgoipiaa oaoaaIe apbv

Iteracja: 7500, Strata: 139.8100

--- GENEROWANY TEKST ---

pgorypnignyhhflmngtttdieae (gml, gogag,iodaacigcettetta,tanntteppapr o pnlaheah
nhuheIehemseahahso mo hl ithlut ihaulainnnmunssenr a(hst,tnt la llpauc-
lallvcestehtlntnn tnt tntllrnrtrrutlttlttcarlhsgn

Iteracja: 8000, Strata: 139.4895

--- GENEROWANY TEKST ---

sf nvtani jAilpgliileytrmaggoe,olggdgh nouloohuhemIaln tvaaatakr lttgl
nkftgnAtallrnxlsltra,totml ansanlansyanranagnlgrscansnaagAnn-
nons(arnaannnprnsnsfaanahsaxnai(racpaTt Caapgaarn(aevAp)Aat (a a

Iteracja: 8500, Strata: 139.5825

--- GENEROWANY TEKST ---

nrmellcvilnisjdr (taazubdadegtAeodbt rdldstnareoaatat olea ejua ea ter
a,otIrannt dmve ,nua gcedavhu tamaaojnhhioiaah ahaenkvagchhmrhge hata-
bavtrhhmx ki o.(lygaarcathrhxchhauharh l hhhshembrahahshnh

Iteracja: 9000, Strata: 140.0914

--- GENEROWANY TEKST ---

oastk ovalsi.(aaodtpteoysaorh dhptg ,aasdgraaugnae fsaieeiiiaeheoege hh ein-
ggedateaei heaaetsetaaama elacealigcaaga ain uaigocve gaagtegtt ae e)gcaggae
agaao)ahatttaaaguueantegouccacanyc iaauaavage

Iteracja: 9500, Strata: 140.2175

--- GENEROWANY TEKST ---

sionidvaricttiaai olsglaaatttf aa toae-aaiehheh saehiuetttu lmeohjmssbupneaa-
nawlm,tbasuotliqvoa bi ostlua aisdfa ,bagaea rtsdeananilaiin t eabeh as lh
hdtaephnlreeaibIeonhalhnapao.aaennuteuss,ia nfeir

Iteracja: 10000, Strata: 139.9178

--- GENEROWANY TEKST ---

rs tiplptiiofa Aaifdtatctceaito,aatetti maoroo al oaamenataiozlaacebna-
tuaaaaaaaaaufaeaaaag eaeheteeaeae o-aea)zgeuauaaeca egagateehteiteeiaefigutdht
atagceotaaatagaieoaatyleteseagaargne cgh(aagefeetage

Trening zakończony po 10000 iteracjach. Ostateczna strata: 139.9178

Link do repozytorium:<https://github.com/Uczelniane/MK.git>

Analiza wyników

Model był trenowany przez **10 000 iteracji** (kroków BPTT)

Metryka	Iteracja 0	Iteracja 8500	Iteracja 10000
Strata	186.8835	139.5825	139.9178

Analiza straty:

- Początkowa strata była wysoka (186.88), co jest typowe dla losowo zainicjowanej sieci.
- Strata gwałtownie spadała do około 8500 iteracji, gdzie osiągnęła lokalne minimum.
- W dalszej fazie (po 8500 iteracji) krzywa uczenia spłaszczyła się i wykazywała niestabilność (wahania, a nawet nieznaczne wzrosty), co sugeruje, że model wykorzystał już niemal całą informację dostępną w małym zbiorze danych.

3. Wnioski

- Zbyt duży model na zbyt małe dane: Ponieważ tekst źródłowy jest bardzo krótki, model szybko "zapamiętał" wszystkie wzorce, a następnie zaczął generować statystycznie najbardziej opłacalne (najczęściej występujące) sekwencje znaków, zamiast uczyć się syntaktyki języka naturalnego
- Brak uogólnienia: Tekst generowany po 8500 iteracjach nie zawiera żadnych prawdziwych słów (poza pojedynczymi literami i częstymi kombinacjami), co potwierdza, że sieć nie była w stanie uogólnić wiedzy poza mikro-wzorce znakowe.